

ROS2 Architecture

Overview

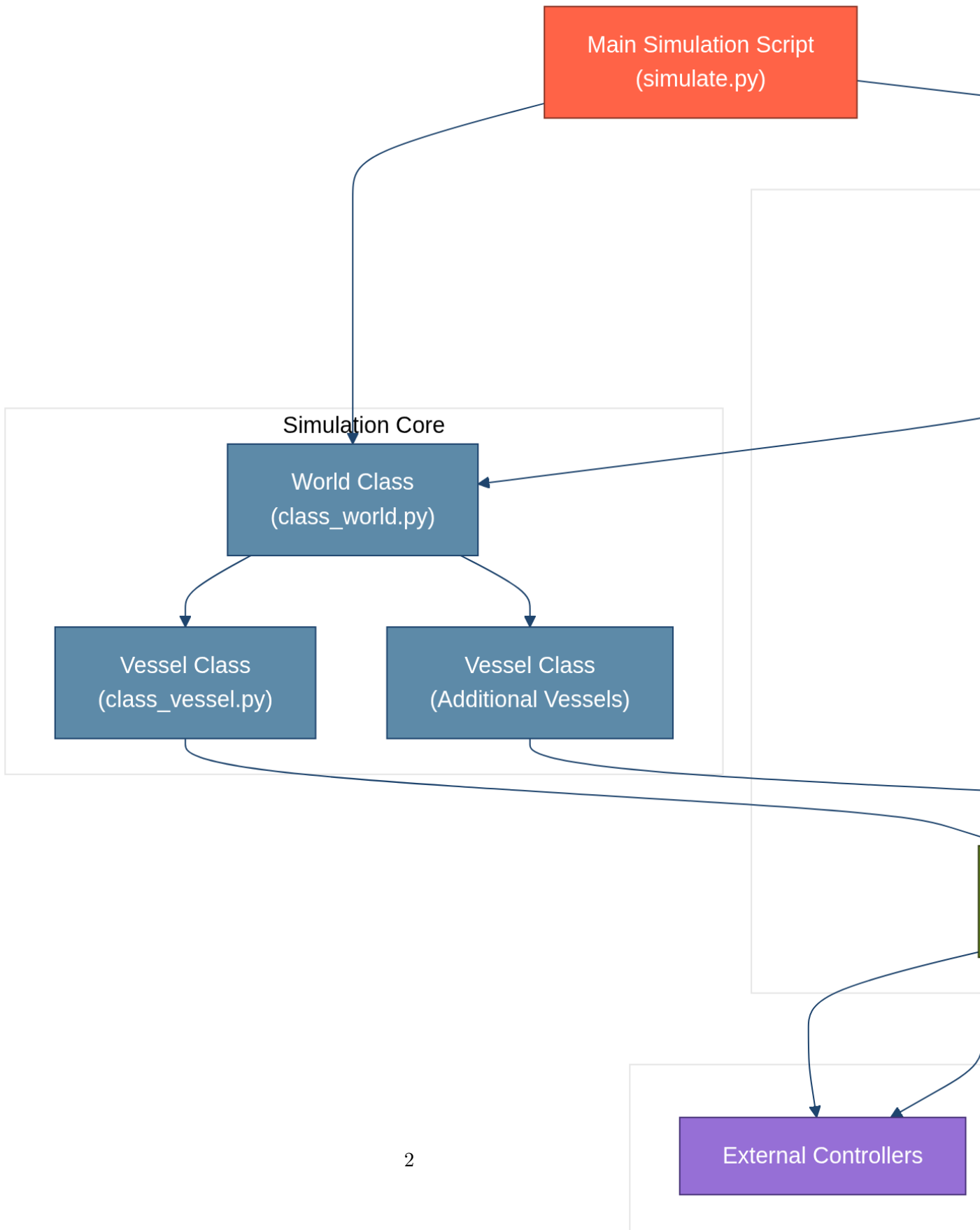
The Panisim runs on the Robot Operating System 2 (ROS2), enabling distributed simulation, external control, visualization, and data recording. The ROS2 integration architecture follows a modular design pattern that separates the simulation core from the communication layer. The Docker container of the simulator runs ROS2 Humble.

Tip

The ROS2 integration allows the Panisim to run completely isolated from controllers or guidance systems, thus enabling you to code your own controllers or guidance systems to interact with the simulator.

Architecture

The ROS2 integration consists of several key components that work together to bridge the simulation with the ROS2 ecosystem:



Explanation of the Architecture Flow:

- **Main Simulation Script (`simulate.py`):** Initiates the simulation by:
 - Calling `rclpy.init()` to initialize the ROS2 middleware
 - Creating a `World` instance by calling `World(config_file_path)`
 - Instantiating a `World_Node` that wraps the simulation in ROS2
 - Calling `world.start_vessel_ros_nodes(world_node)` to connect vessels to ROS2
- **World Class (`class_world.py`):**
 - Maintains multiple vessel instances using `vessels = []`
 - Processes world input with `process_world_input(world_file)`
 - Creates vessels via `Vessel(vessel_params, hydrodynamic_data, vessel_id)`
 - Updates simulation state with `step()` method that calls `vessel.step()` for each vessel
- **World_Node Class (`class_world_node_ros2.py`):**
 - Inherits from ROS2 Node class
 - Creates a timer with `create_timer(1/self.rate, self.world.step)` to drive simulation
- **Vessel_Pub_Sub Class (`class_vessel_pub_sub_ros2.py`):**
 - Creates publishers, subscribers, and timers for each vessel
 - Publishes vessel state and sensor data
 - Handles actuator commands via subscribers

Key Components

Main Simulation Script (`simulate.py`)

The entry point for running Panisim with ROS2 integration. This script:

1. Initializes the ROS2 environment
2. Creates the World simulation instance
3. Sets up the World_Node for ROS2 communication
4. Establishes connections between simulation and ROS2
5. Runs the ROS2 spin loop in a separate thread

```
def main():
    # Creates an object of class 'World'
    rclpy.init()
    world = World('/workspaces/mavlab/inputs/simulation_input.yml')
    world_node = World_Node(world_rate=1/world.dt)

    world.node = world_node
    world.start_vessel_ros_nodes(world_node)

    # Run ROS on a separate thread
    ros_thread_instance = threading.Thread(target=ros_thread, args=(world_node,))
    ros_thread_instance.start()

    # Prints available ROS2 topics and usage information
    # ...
```

The `ros_thread` function is defined as:

```
def ros_thread(node):
    rclpy.spin(node)
    rclpy.shutdown()
```

This function handles the ROS2 event loop, allowing callbacks to be processed in the background while the main thread can perform other tasks.

World Class (`class_world.py`)

The World class serves as the simulation environment container, managing:

- Multiple vessel instances within a unified simulation
- Initialization from configuration files
- Simulation time stepping across all vessels
- Global parameters like GPS datum and world boundaries

```
class World():
    terminate = False           # Flag indicating simulation termination
    vessels = []                # List of Vessel objects
    nvessels = 0                # Number of vessels
    size = np.zeros(3)          # Size of the world (X-Y-Z)
    gps_datum = None            # GPS reference point
    node = None                 # ROS2 node reference
```

```

dt = 0.01                                # Simulation time step

def __init__(self, world_file=None):
    """Initialize the World object from configuration file"""
    if world_file is not None:
        self.process_world_input(world_file)

def step(self):
    """Advance the simulation by one time step"""
    for vessel in self.vessels:
        vessel.step()

```

The `process_world_input` method reads the simulation configuration from YAML files:

```

def process_world_input(self, world_file=None):
    try:
        sim_params, agents = read_input(world_file)
        self.size = np.array(sim_params['world_size'])
        self.nvessels = sim_params['nagents']
        self.gps_datum = np.array(sim_params['gps_datum'])
        agent_count = 0
        self.dt = sim_params['time_step']

        for agent in agents[0:self.nvessels]:
            vessel_config = agent['vessel_config']
            hydrodynamic_data = agent['hydrodynamics']
            self.vessels.append(Vessel(vessel_params=vessel_config,
                                       hydrodynamic_data=hydrodynamic_data,
                                       vessel_id=agent_count))

            agent_count += 1
    except yaml.YAMLError as exc:
        print(exc)
        exit()

```

The `World` class includes a method to start the ROS2 nodes for all vessels:

```

def start_vessel_ros_nodes(self, world_node):
    """Initialize ROS2 nodes for all vessels in the world"""
    for vessel in self.vessels:
        vessel.vessel_node = Vessel_Pub_Sub(vessel, world_node)

```

World_Node Class (class_world_node_ros2.py)

The World_Node class is a ROS2 node wrapper for the World class:

```
class World_Node(Node):
    rate = None
    def __init__(self, world_rate=100, world_file=None):
        super().__init__('world')
        self.rate = world_rate
        self.world = World(world_file)
        self.create_timer(1/self.rate, callback=self.world.step)
```

This class: - Inherits from the ROS2 Node class - Creates a timer that triggers the simulation step function at a specified rate - Provides the ROS2 context for the World class

The `create_timer` method sets up a timer that calls the `World.step()` method at regular intervals determined by `world_rate`. This is what drives the simulation forward in time while maintaining synchronization with the ROS2 ecosystem.

Vessel_Pub_Sub Class (class_vessel_pub_sub_ros2.py)

The Vessel_Pub_Sub class implements the ROS2 communication interface for vessel objects:

```
class Vessel_Pub_Sub():
    def __init__(self, vessel, world_node):
        """Initialize the vessel interface"""
        self.world_node = world_node
        self.vessel = vessel
        self.vessel_id = vessel.vessel_id
        self.vessel_name = vessel.vessel_name
        self.topic_prefix = f'{self.vessel_name}_{self.vessel_id:02d}'

        # Initialize publishers, subscribers, and timers
        # ...
```

This class:

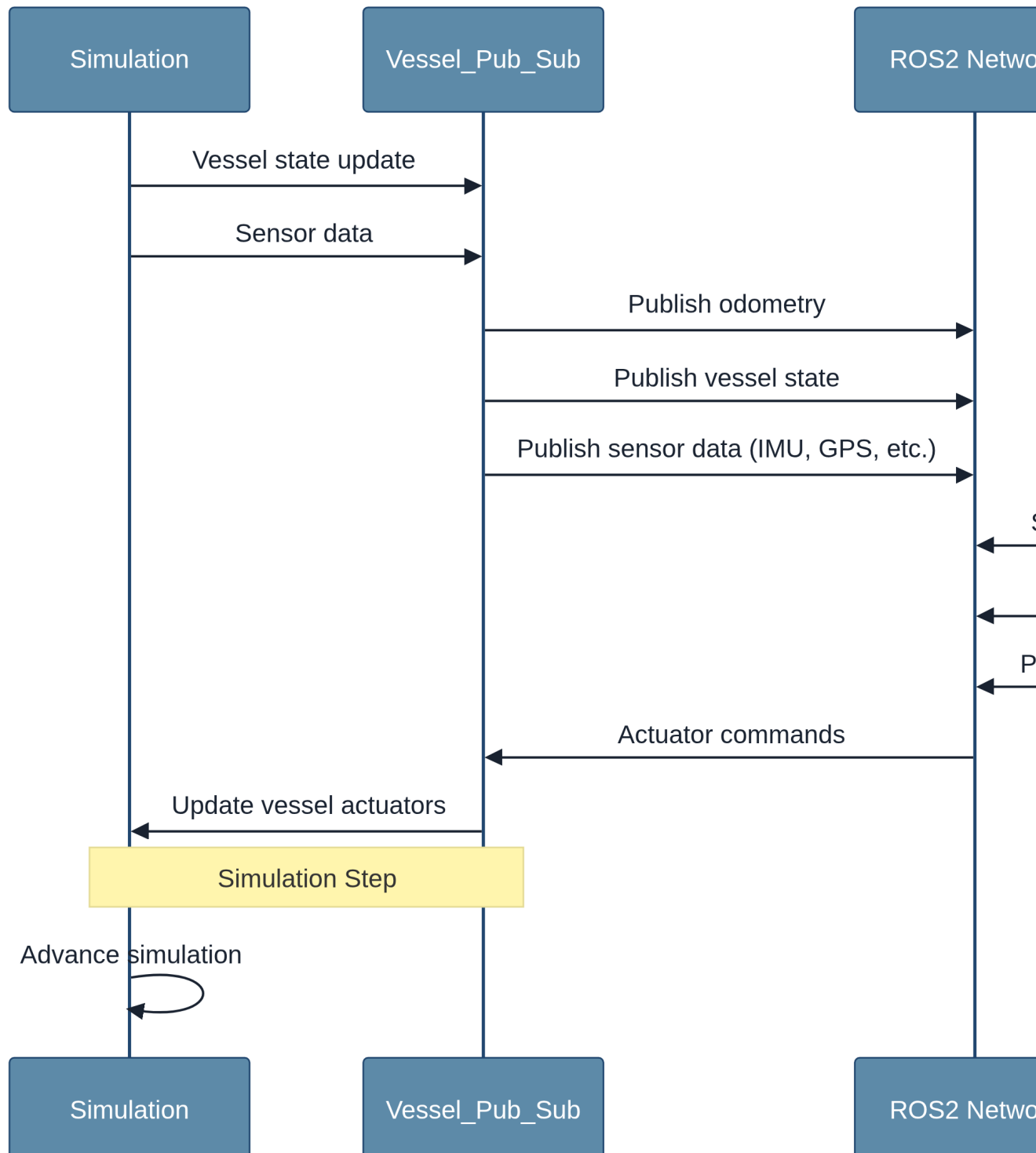
- Creates and manages ROS2 publishers for vessel state and sensor data
- Sets up subscribers for actuator commands
- Handles message conversion between simulator data and ROS2 messages
- Manages sensor data publication at appropriate rates

Key initialization methods include:

- Setting up control surface and thruster mappings
- Creating sensor objects and their publishers
- Initializing odometry and vessel state publishers
- Creating actuator command subscribers

Message Flow

The communication between simulation components and the ROS2 ecosystem follows this message flow:



Detailed Function Call Sequence:

1. Vessel State Update:

- `vessel.step()` in `World.step()` updates the vessel state
- This updates `vessel.current_state`, which is accessed by publishers

2. Sensor Data Generation:

- Sensor objects call `get_measurement()` to generate simulated sensor readings based on vessel state

3. Publishing Data to ROS2:

- `publish_odometry()` is called by a timer to publish vessel position and orientation
- `publish_vessel_state()` is called by a timer to publish complete state vector
- `_publish_sensor()` is called by sensor-specific timers to publish sensor data

4. External Nodes Subscribing:

- External ROS2 nodes subscribe to topics using `ros2 topic echo` or programmatically

5. Control Commands:

- External nodes publish to `/<vessel_name>_<id>/actuator_cmd`
- `actuator_callback(msg)` processes these commands

6. Updating Actuator States:

- `vessel.delta_c` (control surfaces) and `vessel.n_c` (thrusters) are updated
- These affect the vessel dynamics in the next simulation step

7. Simulation Step:

- The timer in `World_Node` triggers `world.step()`
- This advances all vessels' states using their dynamics models

Topics and Messages

Each vessel in the simulation publishes data on several ROS2 topics:

Standard Topics

Topic	Message Type	Description
<code>/<vessel_name>_<id>/odometry</code>	<code>nav_msgs/Odometry</code>	Vessel position, orientation, and velocities

Topic	Message Type	Description
/<vessel_name>_<id>/vessel_state	std_msgs/Float64MultiArray	Complete vessel state including actuators
/<vessel_name>_<id>/interfaces/Actuator	interfaces/Actuator	Commands for control surfaces and thrusters

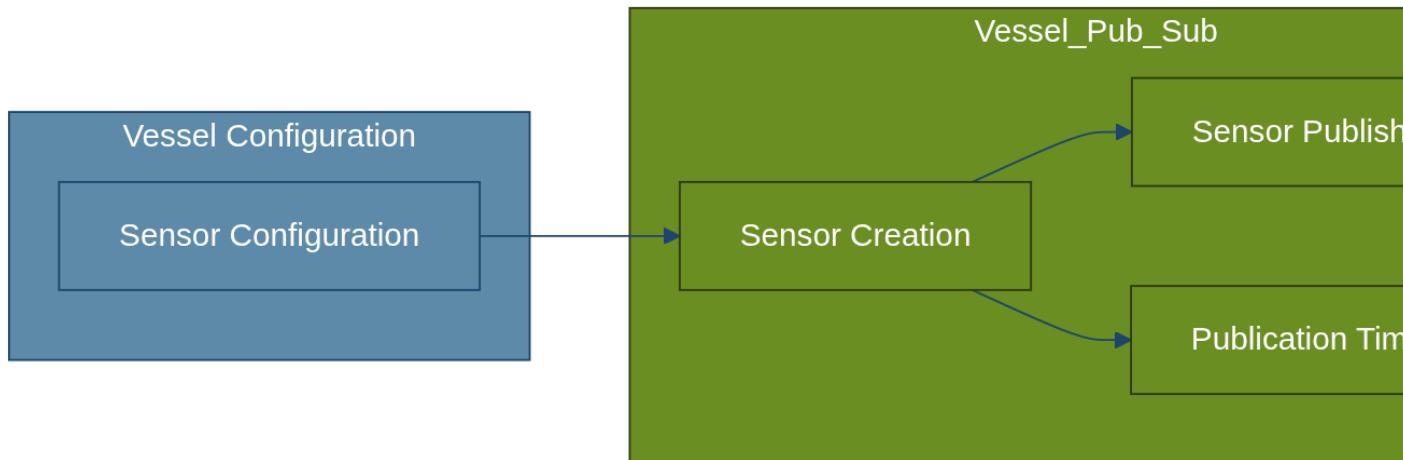
Sensor Topics

Each sensor configured for a vessel publishes on its own topic:

Sensor	Topic	Message Type
IMU	/<vessel_name>_<id>/sensor_msgs/Imu	sensor_msgs/Imu
GPS	/<vessel_name>_<id>/sensor_msgs/NavSatFix	sensor_msgs/NavSatFix
DVL	/<vessel_name>_<id>/interfaces/DVL	interfaces/DVL
UWB	/<vessel_name>_<id>/geometry_msgs/PoseWithCovarianceStamped	geometry_msgs/PoseWithCovarianceStamped

Sensor Integration

Sensors are created and managed through the Vessel_Pub_Sub class:



Detailed Function Calls in Sensor Integration:

1. SensorConfig to SensorCreation:

- During `Vessel_Pub_Sub` initialization, it reads sensor configurations from `vessel.vessel_config['sensors']`
- For each sensor configuration, it calls `create_sensor(sensor_config, vessel_id, topic_prefix, self)`

2. SensorCreation to Publishers and Timers:

- After creating a sensor object, `Vessel_Pub_Sub` creates a publisher using:

```
self.world_node.create_publisher(self._get_msg_type(sensor.sensor_type), sensor_topi
```

- It also creates a timer for each sensor:

```
self.world_node.create_timer(1/sensor.rate, lambda s=sensor: self._publish_sensor(s
```

3. Timer to PublishSensor:

- Each timer periodically calls `_publish_sensor(sensor)` at the sensor's configured rate

4. PublishSensor to GetMeasurement:

- The `_publish_sensor` method calls `sensor.get_measurement()` to obtain the latest sensor reading
- Each sensor type has its own implementation of `get_measurement()`

5. PublishSensor to CreateMessage:

- After getting the measurement, `_publish_sensor` calls `_create_sensor_message(sensor_type, measurement)`
- This converts the measurement to the appropriate ROS2 message type

6. CreateMessage to Topics:

- Finally, the message is published to the ROS2 network:

```
s['pub'].publish(msg)
```

The process for sensor integration:

1. Sensor configurations are defined in the vessel parameters (input files `sensors.yml`)
2. The `Vessel_Pub_Sub` class creates sensor objects using the `create_sensor` factory function
3. For each sensor, a publisher and timer are created
4. The timer periodically calls the sensor's `get_measurement()` method
5. Measurements are converted to ROS2 messages and published

Key methods in the sensor integration:

```
def _publish_sensor(self, sensor):
    """Publish sensor data."""
    measurement = sensor.get_measurement()
    msg = self._create_sensor_message(sensor.sensor_type, measurement)

    # Find the publisher for this sensor
    for s in self.sensors:
        if s['sensor'] == sensor:
            s['pub'].publish(msg)
            break

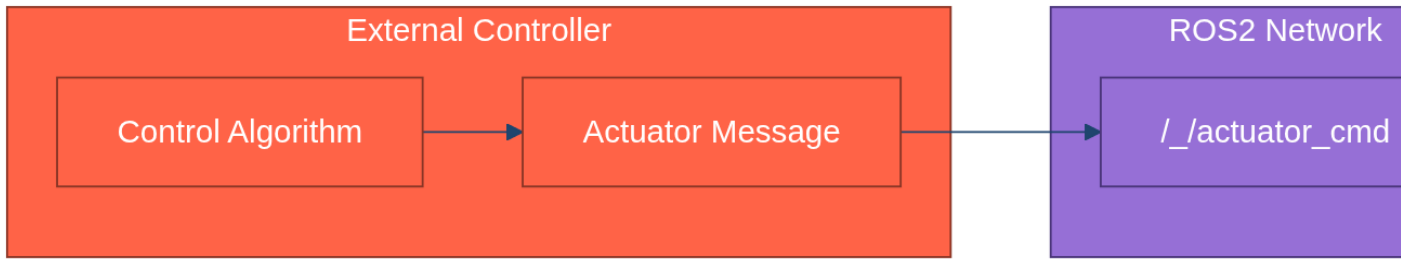
def _create_sensor_message(self, sensor_type, measurement):
    """Create a ROS message for sensor data."""
    current_time = self.world_node.get_clock().now().to_msg()

    # Create appropriate message based on sensor type
    if sensor_type == 'IMU':
        msg = Imu()
        # Fill message fields
    elif sensor_type == 'GPS':
        msg = NavSatFix()
        # Fill message fields
    # ... other sensor types

    return msg
```

Actuator Control

Vessels can be controlled through ROS2 by sending actuator commands:



Detailed Function Calls in Actuator Control:

1. ControlAlgorithm to ActuatorMsg:

- External control algorithms compute desired actuator values
- They create an `Actuator` message with the desired values

2. ActuatorMsg to ActuatorTopic:

- The controller publishes the message to the actuator command topic
- This is done using ROS2's `publisher.publish(msg)` mechanism

3. ActuatorTopic to Subscriber:

- The subscriber in `Vessel_Pub_Sub` receives the message
- This was set up during initialization with:

```
self.actuator_sub = self.world_node.create_subscription(
    Actuator,
    f'{self.topic_prefix}/actuator_cmd',
    self.actuator_callback,
    1
)
```

4. Subscriber to Callback:

- The subscriber triggers `actuator_callback(msg)` in `Vessel_Pub_Sub`

5. Callback to ControlSurfaces and Thrusters:

- The callback parses the message and updates vessel control values:

```
# For control surfaces
self.vessel.delta_c[idx] = value * np.pi / 180.0

# For thrusters
self.vessel.n_c[idx] = value
```

Actuator Message Structure (Actuator.msg)

The `Actuator.msg` is a custom message type used for controlling vessel actuators. Its structure is:

```
std_msgs/Header header      # Standard header with timestamp
string[] actuator_names     # Array of actuator identifiers
float64[] actuator_values   # Array of corresponding values
```

The actuator names use a prefixing convention: - **cs**_N for control surfaces (e.g., **cs**_1 for the first control surface) - **th**_N for thrusters (e.g., **th**_1 for the first thruster)

Values for control surfaces are specified in **degrees** (converted to radians internally) and values for thrusters are in **RPM** (Revolutions Per Minute).

Example usage in `class_vessel_pub_sub_ros2.py`:

```
def actuator_callback(self, msg):
    """Handle actuator command messages."""
    if len(msg.actuator_names) != len(msg.actuator_values):
        self.world_node.get_logger().warn('Mismatch between actuator IDs and values length')
        return

    for actuator_id, value in zip(msg.actuator_names, msg.actuator_values):
        try:
            if actuator_id.startswith('cs_'):
                # Handle control surface (convert degrees to radians)
                if actuator_id in self.control_surface_ids:
                    idx = self.control_surface_ids[actuator_id]
                    self.vessel.delta_c[idx] = value * np.pi / 180.0
                else:
                    self.world_node.get_logger().warn(f'Unknown control surface ID: {actuator_id}')
            elif actuator_id.startswith('th_'):
                # Handle thruster (RPM value)
```

```

        if actuator_id in self.thruster_ids:
            idx = self.thruster_ids[actuator_id]
            self.vessel.n_c[idx] = value
        else:
            self.world_node.get_logger().warn(f'Unknown thruster ID: {actuator_id}')

    else:
        self.world_node.get_logger().warn(f'Invalid actuator ID format: {actuator_id}')
except (ValueError, IndexError):
    self.world_node.get_logger().warn(f'Unknown actuator ID: {actuator_id}')

```

During initialization, `Vessel_Pub_Sub` builds mappings between actuator IDs (as defined in `control_surfaces.yml` and `thrusters.yml`) and their indices:

```

# Store actuator IDs with type prefixes
self.control_surface_ids = {} # Maps 'cs_id' to index
self.thruster_ids = {}      # Maps 'th_id' to index

for idx, cs in enumerate(self.control_surfaces):
    cs_id = cs.get('control_surface_id', idx+1)
    self.control_surface_ids[f'cs_{cs_id}'] = idx

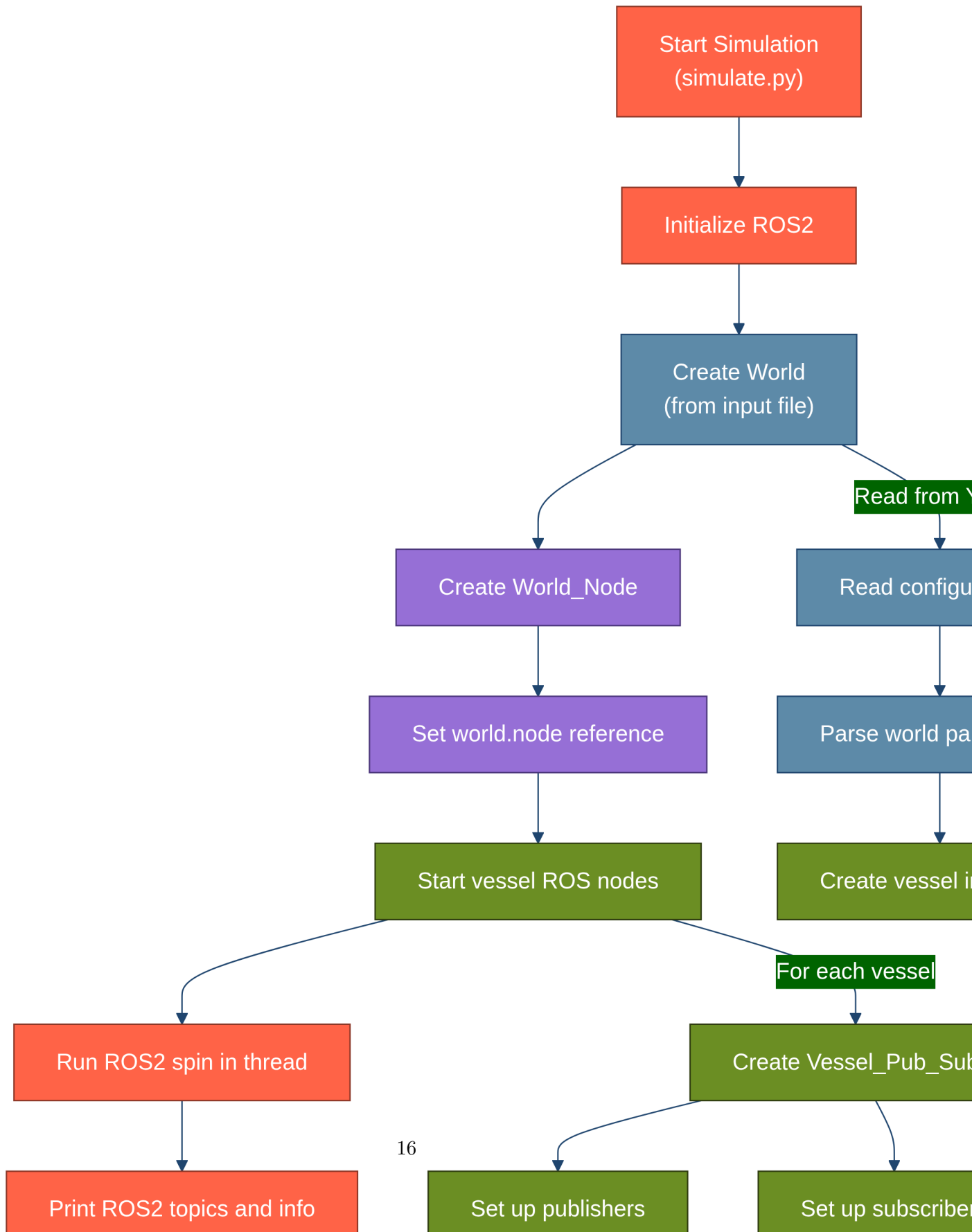
for idx, th in enumerate(self.thrusters):
    th_id = th.get('thruster_id', idx+1)
    self.thruster_ids[f'th_{th_id}'] = idx

```

These mappings ensure that actuator commands are properly routed to the correct control surfaces and thrusters in the vessel's internal state.

World and Vessel Population

The process of populating the world with vessels follows these steps:



Detailed Function Calls in World and Vessel Population:

1. Start to Init:

- `simulate.py` starts and calls `rclpy.init()` to initialize the ROS2 middleware
- This initializes the ROS2 context for the application

2. Init to CreateWorld:

- `World('/workspaces/mavlab/inputs/simulation_input.yml')` is called
- This creates the World instance and loads configuration

3. CreateWorld to ReadConfig:

- `World.__init__` calls `self.process_world_input(world_file)`
- `process_world_input` calls `read_input(world_file)` to parse YAML

4. ReadConfig to ParseWorld:

- `process_world_input` extracts world parameters like:
 - `self.size = np.array(sim_params['world_size'])`
 - `self.nvessels = sim_params['nagents']`
 - `self.gps_datum = np.array(sim_params['gps_datum'])`

5. ParseWorld to CreateVessels:

- `process_world_input` creates vessel instances:

```
self.vessels.append(Vessel(vessel_params=vessel_config,
                           hydrodynamic_data=hydrodynamic_data,
                           vessel_id=agent_count))
```

6. CreateWorld to CreateNode:

- `World_Node(world_rate=1/world.dt)` is called
- This creates a ROS2 node wrapper for the World

7. SetNode to StartVessels:

- `world.start_vessel_ros_nodes(world_node)` is called
- This iterates through vessels and creates ROS2 interfaces

8. StartVessels to CreateVesselPubSub:

- `Vessel_Pub_Sub(vessel, world_node)` is called for each vessel
- Each vessel gets its own ROS2 communication interface

9. CreateVesselPubSub to SetupPublishers:

- Publishers are created for each vessel's odometry and state:

```

self.odometry = {
    'pub': self.world_node.create_publisher(Odometry, f'{self.topic_prefix}/odometry', 10),
    'timer': self.world_node.create_timer(self.vessel.vessel_config['time_step'], self.odometry_callback)
}

```

10. CreateVesselPubSub to SetupSubscribers:

- Subscribers are created for actuator commands:

```

self.actuator_sub = self.world_node.create_subscription(
    Actuator,
    f'{self.topic_prefix}/actuator_cmd',
    self.actuator_callback,
    1
)

```

11. CreateVesselPubSub to SetupSensors:

- Sensors are created and configured based on vessel configuration:

```

sensor = create_sensor(sensor_config, self.vessel_id, self.topic_prefix, self)
self.sensors.append({
    'sensor': sensor,
    'pub': self.world_node.create_publisher(self._get_msg_type(sensor.sensor_type), f'{self.topic_prefix}/sensor/{self.vessel_id}', 10),
    'timer': self.world_node.create_timer(1/sensor.rate, lambda s=sensor: self._publish_sensor(s))
})

```

12. StartVessels to SpinThread:

- `threading.Thread(target=ros_thread, args=(world_node,))` is created and started
- This runs `rospy.spin(node)` in a separate thread

13. SpinThread to PrintInfo:

- Available ROS2 topics are listed and helpful usage information is printed

Key steps in this process:

1. The simulation begins in `simulate.py`
2. ROS2 is initialized
3. A World object is created from the input configuration file
4. A World_Node is created and linked to the World object
5. Vessel ROS nodes are started for each vessel in the world
6. For each vessel, a Vessel_Pub_Sub object is created, which:
 - Sets up publishers for odometry and vessel state

- Sets up subscribers for actuator commands
- Creates sensor objects based on the vessel configuration
- Sets up timers for publishing sensor data

Running the Simulation

To run the MAV simulator with ROS2 integration:

```
# Navigate to the workspace
cd /path/to/makara

# Build ROS2 packages
colcon build

# Source the workspace
source install/setup.bash

# Run the simulator
ros2 run mav_simulator simulate
```

After starting the simulation, you can interact with it using standard ROS2 commands:

```
# List all available topics
ros2 topic list

# Subscribe to vessel odometry
ros2 topic echo /<vessel_name>_<id>/odometry_sim

# Publish control commands
ros2 topic pub /<vessel_name>_<id>/actuator_cmd interfaces/Actuator \
"{header: {stamp: {sec: 0}}, actuator_names: ['cs_1', 'th_1'], actuator_values: [15.0, 1200.0]}
```

Best Practices

- **Launch Files:** Create launch files for different simulation scenarios. There are some example launch files in the `/workspace/mavlab/ros2_ws/src/mav_simulator/mav_simulator/launch` folder.